

Implementasi Principal Component Analysis (PCA) untuk Klasifikasi Breast Cancer

1 Pendahuluan

Principal Component Analysis (PCA) adalah teknik reduksi dimensi yang mengubah fitur-fitur berkorelasi menjadi komponen principal yang tidak berkorelasi. Praktikum ini mengimplementasikan PCA pada dataset Breast Cancer untuk mengurangi 30 fitur menjadi 3 komponen principal, kemudian membandingkan performa klasifikasi SVM dengan dan tanpa PCA.

2 Dataset

Dataset Breast Cancer memiliki karakteristik:

- Total sampel: 569 data
- Fitur: 30 fitur numerik (mean, standard error, worst values dari 10 karakteristik sel)
- Target: Diagnosis (M=Malignant/ganas, B=Benign/jinak)
- File: breast_cancer.csv

3 Implementasi

3.1 Import Library

```
1 import numpy as np
2 import pandas as pd
3 from sklearn.model_selection import train_test_split
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.decomposition import PCA
6 from sklearn.svm import SVC
7 from sklearn.metrics import accuracy_score,
  classification_report, confusion_matrix
8 import matplotlib.pyplot as plt
9 from mpl_toolkits.mplot3d import Axes3D
```

Listing 1: Import Library yang Diperlukan

Penjelasan: Import library untuk manipulasi data (pandas, numpy), machine learning (sklearn), dan visualisasi (matplotlib).

3.2 Load dan Eksplorasi Data

```
1 df = pd.read_csv('../Data/breast_cancer.csv')
2 print(f"Dataset shape: {df.shape}")
3 df.head()
```

Listing 2: Load Dataset

Output:

Dataset shape: (569, 33)

Dataset memiliki 569 baris dan 33 kolom (termasuk id, diagnosis, 30 fitur, dan 1 kolom sampah "Unnamed: 32").

```
1 df.info()
2 df.describe()
```

Listing 3: Informasi Dataset

Penjelasan: `df.info()` menampilkan tipe data dan jumlah non-null values. `df.describe()` menampilkan statistik deskriptif (mean, std, min, max, quartiles) untuk setiap fitur numerik.

```
1 print("Kelas Diagnosis:")
2 df['diagnosis'].value_counts()
```

Listing 4: Distribusi Kelas Target

Output:

```
B    357
M    212
```

Dataset memiliki 357 kasus Benign (jinak) dan 212 kasus Malignant (ganas).

3.3 Data Cleaning dan Preprocessing

```
1 # Hapus kolom yang SEMUA nilainya NaN
2 df_clean = df.dropna(axis=1, how='all')
3 print(f"Setelah hapus kolom sampah: {df_clean.shape}")
4
5 # Hapus baris dengan missing values (jika ada)
6 df_clean = df_clean.dropna()
7
8 # Pisahkan fitur dan label
9 X = df_clean.drop(['id', 'diagnosis'], axis=1).values
10 y = df_clean['diagnosis'].map({'M': 1, 'B': 0}).values
11
12 print("Shape X : ", X.shape)
13 print("Shape y : ", y.shape)
14 print("Jumlah kelas Malignant (1):", np.sum(y == 1))
15 print("Jumlah kelas Benign (0):", np.sum(y == 0))
```

Listing 5: Pembersihan Data

Output:

Setelah hapus kolom sampah: (569, 32)

Shape X : (569, 30)

Shape y : (569,)

Jumlah kelas Malignant (1): 212

Jumlah kelas Benign (0): 357

Penjelasan:

- `dropna(axis=1, how='all')`: Menghapus kolom yang semua nilainya NaN (kolom "Unnamed: 32")
- Drop kolom 'id' dan 'diagnosis' untuk mendapatkan matriks fitur X
- Mapping diagnosis: M=1, B=0
- Hasil: 569 sampel dengan 30 fitur

3.4 Split Data dan Standardisasi

```
1 X_train, X_test, y_train, y_test = train_test_split(  
2     X, y, test_size=0.2, random_state=42, stratify=y  
3 )  
4  
5 print("Shape X_train : ", X_train.shape)  
6 print("Shape X_test : ", X_test.shape)
```

Listing 6: Train-Test Split

Output:

Shape X_train : (455, 30)

Shape X_test : (114, 30)

Penjelasan: Data dibagi 80:20 dengan stratifikasi untuk menjaga proporsi kelas. Data training: 455 sampel, data testing: 114 sampel.

```
1 scaler = StandardScaler()  
2 X_train_scaled = scaler.fit_transform(X_train)  
3 X_test_scaled = scaler.transform(X_test)
```

Listing 7: Standardisasi Fitur

Penjelasan: `StandardScaler` mengubah fitur menjadi mean=0 dan std=1. Penting untuk PCA karena PCA sensitif terhadap skala fitur.

3.5 Model SVM Tanpa PCA (Baseline)

```

1 svm_no_pca = SVC(kernel='rbf', gamma='scale', random_state
    =42)
2 svm_no_pca.fit(X_train_scaled, y_train)
3
4 y_pred_no_pca = svm_no_pca.predict(X_test_scaled)
5 accuracy_no_pca = accuracy_score(y_test, y_pred_no_pca)
6 print("Accuracy tanpa PCA: ", accuracy_no_pca)
7 print("\nClassification Report tanpa PCA:\n",
8       classification_report(y_test, y_pred_no_pca))

```

Listing 8: Training SVM pada Data Original

Output:

Accuracy tanpa PCA: 0.9736842105263158

	precision	recall	f1-score	support
0	0.96	1.00	0.98	72
1	1.00	0.93	0.96	42
accuracy			0.97	114
macro avg	0.98	0.96	0.97	114
weighted avg	0.98	0.97	0.97	114

Penjelasan: SVM dengan kernel RBF pada 30 fitur original mencapai akurasi **97.37%**. Model baseline ini sangat baik dengan precision dan recall tinggi untuk kedua kelas.

3.6 Implementasi PCA

```

1 pca = PCA(n_components=3)
2 X_train_pca = pca.fit_transform(X_train_scaled)
3 X_test_pca = pca.transform(X_test_scaled)
4
5 print("Shape X_train_pca : ", X_train_pca.shape)
6 print("Shape X_test_pca : ", X_test_pca.shape)

```

Listing 9: Reduksi Dimensi dengan PCA

Output:

Shape X_train_pca : (455, 3)

Shape X_test_pca : (114, 3)

Penjelasan: PCA mereduksi 30 fitur menjadi 3 komponen principal. Dimensi berkurang 90% (dari 30 menjadi 3).

```
1 explained_variance = pca.explained_variance_ratio_  
2 print(" Explained Variance Ratio oleh masing-masing  
   komponen PCA:",  
3       explained_variance )  
4 print(" Total Explained Variance oleh 3 komponen PCA:",  
5       np.sum(explained_variance ))
```

Listing 10: Explained Variance Ratio

Output:

Explained Variance Ratio: [0.44272026 0.18971182 0.09393163]

Total Explained Variance: 0.7263637

Penjelasan:

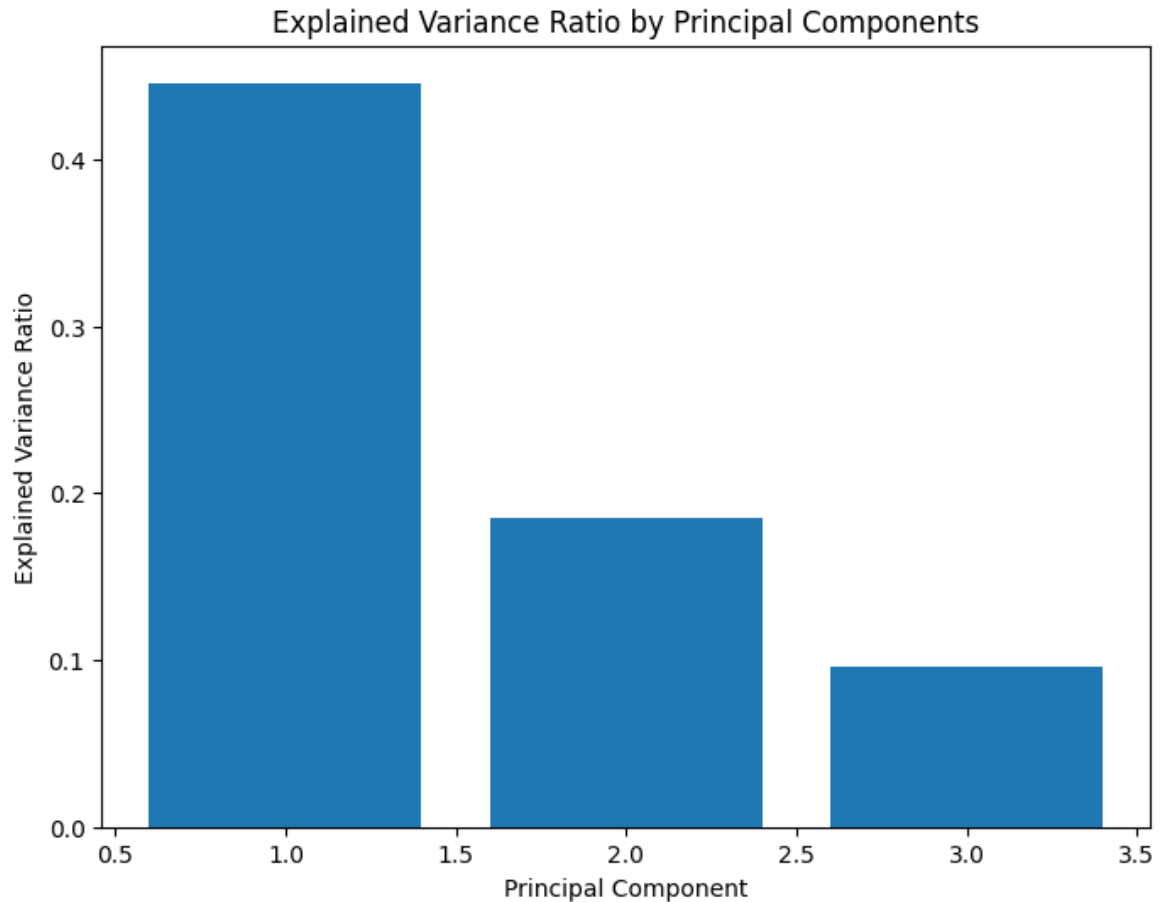
- PC1 menjelaskan 44.27% variansi total
- PC2 menjelaskan 18.97% variansi total
- PC3 menjelaskan 9.39% variansi total
- Total 3 komponen menjelaskan 72.64% variansi data original

3.7 Visualisasi Explained Variance

```
1 plt.bar([1, 2, 3], explained_variance )  
2 plt.xlabel('Principal Component')  
3 plt.ylabel('Explained Variance Ratio')  
4 plt.title('Explained Variance Ratio by Principal  
   Components')  
5 plt.show()
```

Listing 11: Bar Chart Explained Variance

Output Visual:



Bar chart menampilkan 3 batang dengan tinggi menurun: PC1 (44%), PC2 (19%), PC3 (9%). Visualisasi ini menunjukkan bahwa PC1 dominan dalam menjelaskan variansi data.

3.8 Model SVM dengan PCA

```
1 svm_pca = SVC(kernel='rbf', gamma='scale', random_state=42)
2 svm_pca.fit(X_train_pca, y_train)
3
4 y_pred_pca = svm_pca.predict(X_test_pca)
5 accuracy_pca = accuracy_score(y_test, y_pred_pca)
6 print("Accuracy dengan PCA: ", accuracy_pca)
7 print("\nClassification Report dengan PCA:\n",
8       classification_report(y_test, y_pred_pca))
```

Listing 12: Training SVM pada Data PCA

Output:

Accuracy dengan PCA: 0.956140350877193

	precision	recall	f1-score	support
0	0.94	1.00	0.97	72
1	1.00	0.88	0.94	42
accuracy			0.96	114
macro avg	0.97	0.94	0.95	114
weighted avg	0.96	0.96	0.96	114

Penjelasan: SVM dengan 3 komponen PCA mencapai akurasi **95.61%**. Meskipun sedikit menurun dari baseline (97.37%), penurunan hanya 1.76% dengan pengurangan dimensi 90%.

3.9 Visualisasi 3D PCA

```

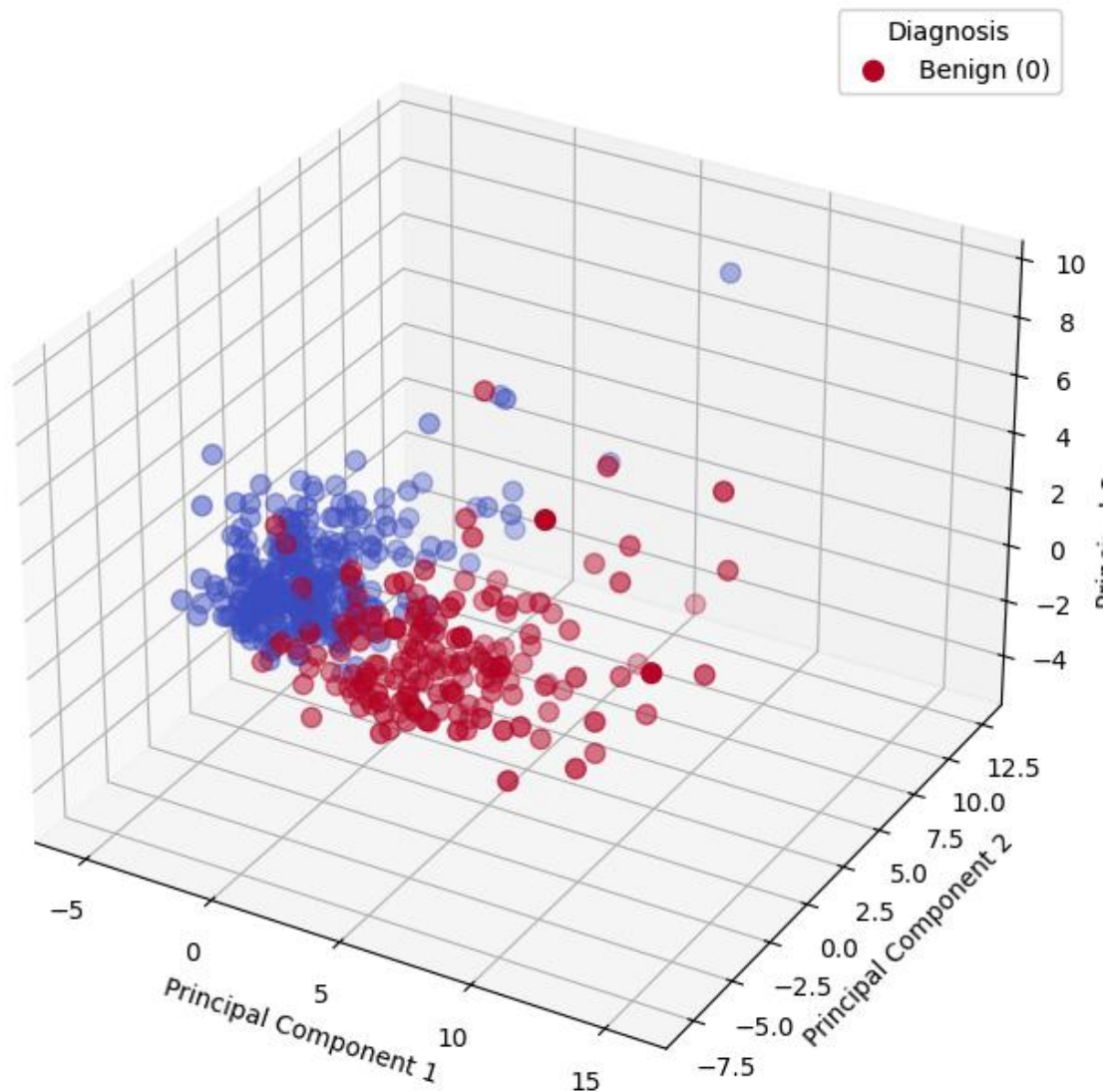
1 fig = plt.figure(figsize=(10,8))
2 ax = fig.add_subplot(111, projection='3d')
3
4 scatter = ax.scatter(
5     X_train_pca[:, 0], X_train_pca[:, 1], X_train_pca[:,
6     2],
7     c=y_train, cmap='coolwarm', s=60
8 )
9 ax.set_title('3D Scatter Plot of PCA-transformed Breast
10 Cancer Data')
11 ax.set_xlabel('Principal Component 1')
12 ax.set_ylabel('Principal Component 2')
13 ax.set_zlabel('Principal Component 3')
14 legend1 = ax.legend(*scatter.legend_elements(),
15                     labels=['Benign (0)', 'Malignant (1)'],
16                     title="Diagnosis")
17 ax.add_artist(legend1)
18 plt.show()

```

Listing 13: 3D Scatter Plot Data PCA

Output Visual:

3D Scatter Plot of PCA-transformed Breast Cancer Data



Scatter plot 3D menampilkan data training dalam ruang 3 komponen principal. Warna biru (Benign) dan merah (Malignant) menunjukkan se- parasi kelas yang jelas. Kedua kelas dapat dibedakan dengan baik dalam ruang PCA 3D, menjelaskan mengapa SVM masih mencapai akurasi tinggi meskipun dimensi berkurang drastis.

3.10 Perbandingan Hasil

```

1 comparison = pd.DataFrame({
2     'Model': ['SVM tanpa PCA', 'SVM dengan PCA'],
3     'Jumlah Fitur': [X_train.shape[1], X_train_pca.shape
4         [1]],
5     'Akurasi': [accuracy_no_pca, accuracy_pca],
6     'Variansi Total PCA' : [None, np.sum(
7         explained_variance)]
8 })
9 comparison

```

Listing 14: Tabel Perbandingan

Output:

Tabel 1: Perbandingan Model SVM dengan dan tanpa PCA

Model	Jumlah Fitur	Akurasi	Variansi PCA
SVM tanpa PCA	30	0.9737	-
SVM dengan PCA	3	0.9561	0.7264

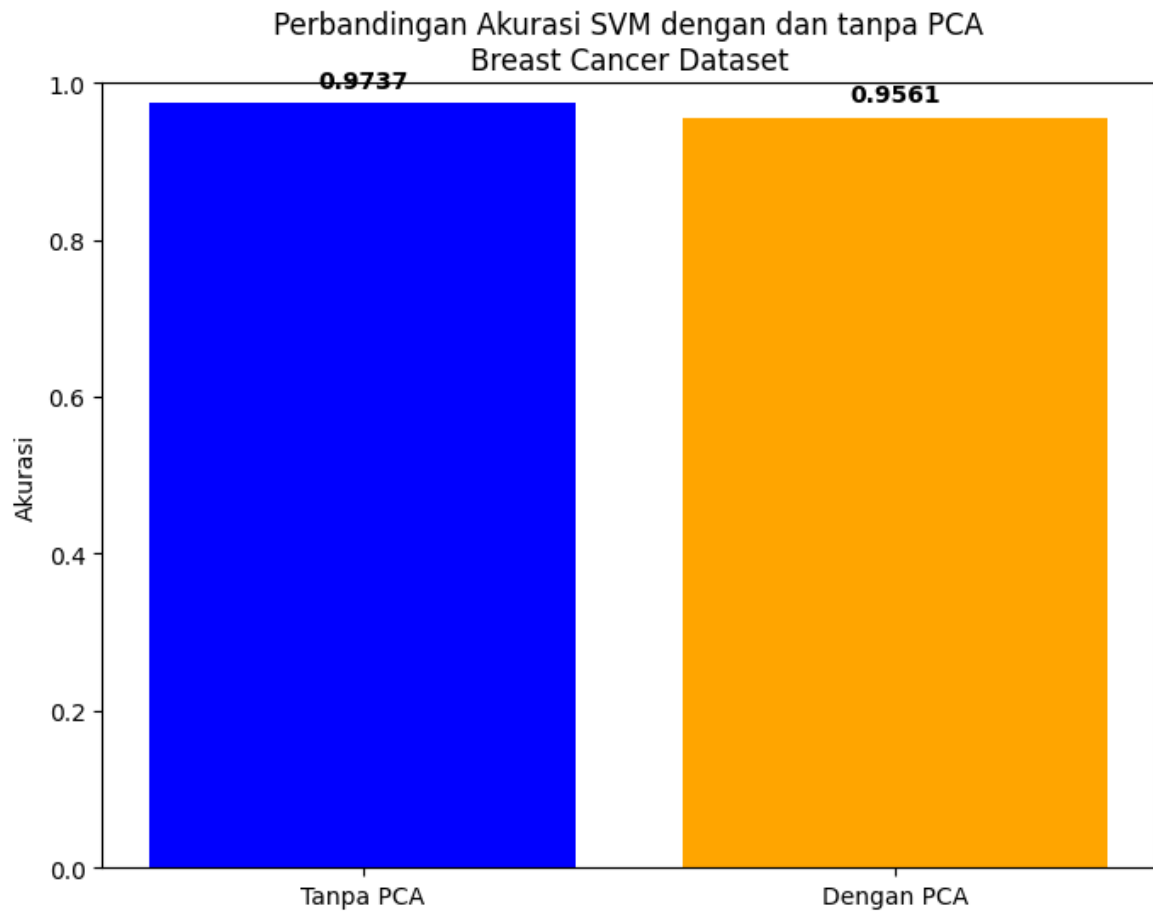
```

1 plt.figure(figsize=(8,6))
2 plt.bar(['Tanpa PCA', 'Dengan PCA'],
3         [accuracy_no_pca, accuracy_pca],
4         color=['blue', 'orange'])
5 plt.title('Perbandingan Akurasi SVM dengan dan tanpa PCA\
6     nBreast Cancer Dataset')
7 plt.ylabel('Akurasi')
8 plt.ylim(0, 1)
9 for i, v in enumerate([accuracy_no_pca, accuracy_pca]):
10     plt.text(i, v + 0.02, f"{v:.4f}", ha='center',
11             fontweight='bold')
12 plt.show()

```

Listing 15: Visualisasi Perbandingan Akurasi

Output Visual:



Bar chart dengan 2 batang vertikal menunjukkan akurasi. Batang biru (Tanpa PCA) di 97.37% sedikit lebih tinggi dari batang oranye (Dengan PCA) di 95.61%. Label nilai ditampilkan di atas setiap batang.

3.11 Confusion Matrix

```
1 cm_no_pca = confusion_matrix(y_test, y_pred_no_pca)
2 print("Confusion Matrix (tanpa PCA):")
3 print(cm_no_pca)
4
5 cm_pca = confusion_matrix(y_test, y_pred_pca)
6 print("\nConfusion Matrix (dengan PCA):")
7 print(cm_pca)
```

Listing 16: Confusion Matrix Kedua Model

Output:

Confusion Matrix (tanpa PCA):

```
[[72  0]
 [ 3 39]]
```

Confusion Matrix (dengan PCA):

```
[[72  0]
 [ 5 37]]
```

Penjelasan:

- **Tanpa PCA:** 72 TN, 0 FP, 3 FN, 39 TP. Sempurna untuk kelas Benign (tidak ada False Positive), 3 Malignant salah klasifikasi sebagai Benign.
- **Dengan PCA:** 72 TN, 0 FP, 5 FN, 37 TP. Tetap sempurna untuk kelas Benign, tapi 5 Malignant salah klasifikasi (naik 2 dari tanpa PCA).
- Perbedaan kecil ini konsisten dengan penurunan akurasi 1.76%

4 Kesimpulan

1. PCA berhasil mereduksi dimensi dari 30 fitur menjadi 3 komponen principal (reduksi 90%) dengan tetap mempertahankan 72.64% variansi data original.
2. Model SVM tanpa PCA mencapai akurasi 97.37%, sementara SVM dengan PCA mencapai 95.61%. Penurunan akurasi hanya 1.76%.
3. Trade-off antara efisiensi komputasi dan akurasi sangat menguntungkan: pengurangan 90% dimensi dengan penurunan akurasi kurang dari 2%.
4. Visualisasi 3D menunjukkan separasi kelas yang jelas dalam ruang PCA, menjelaskan mengapa model dengan hanya 3 fitur masih berkinerja sangat baik.
5. PCA sangat efektif untuk dataset Breast Cancer dan dapat direkomendasikan untuk aplikasi yang memerlukan efisiensi komputasi tanpa mengorbankan performa secara signifikan.